Society for Computer Technology & Research's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

Web Technology Lab [310257]

Mini-Project Report on

Blog Platform

Submitted by

Nandini Deshmukh	31116
Siddhi Chavan	31112

Class: - T.E (2019 course) Branch: -Computer Engineering

ACADEMIC YEAR: 2025-26

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

At the dawn of the new era of digitalization, traditional blogs are evolving into sophisticated content creation tools that can be used not only for personal purposes but also for education, business purposes, and marketing. In general, blogs serve as a way to share knowledge with others and build up a reputation as a specialist or professional who knows a lot about a particular topic or field. Thus, a blog platform is a complex system that consists of a powerful content management tool and is supported by efficient backend infrastructure and smooth functionality.

The current mini-project entitled ”Narrative Blog Platform” seeks to create a fully featured application capable of delivering an engaging experience to the end-user through an intuitive blog application development process. This platform seeks to address essential functions expected in a modern blog system, such as secure user authentication, CRUDD capabilities, live liking capability, and comments management, where one can reply to existing comments.

For maximum efficiency and scalability, the current project utilizes modern technologies that will enable the successful implementation of the project goals. For the frontend, the project utilizes Next.js and Typescript. In the case of the backend, FastAPI is utilized, which is a modern Python-based web framework. In particular, much attention has been devoted to addressing the N+1 problem, which refers to poor performance in database query processes, and also dynamic routing to access users' blogs and individual blog posts.

For the sake of consistency and easy deployment of the platform to any machine, both the frontend and backend parts of the application were containerized using Docker and Docker Compose.

The main features incorporated into the Narrative Blog Platform are as follows:

- **User Authentication**: Registration and login of users and handling sessions, which ensures that all actions that require authentication such as creating, updating, or deleting something can only be done by users who have authenticated themselves.

- **Blogging Interface**: A full interface that allows users to carry out CRUD (Create, Read, Update, Delete) operations on their blog posts and even add images to their blog posts.
- **Likes Feature**: A cool functionality that enables users to like or unlike posts as well as track the number of likes made per post. It will not allow any user to give multiple likes to a particular post.
- **Comments Feature**: Allows users to make replies and comments to comments and posts from other users. This will create room for discussion.
- **Upload Images**: Capability to upload images related to posts or comments. The images will be stored in the backend.
- **Docker Deployment**: Full integration using docker-compose.yml file to run both services (frontend and backend) together.

CHAPTER 2
MOTIVATION

Factors motivating the construction of the Narrative Blog Platform include the following:

1. **The Desire for a Modern Blog Platform**
   Current blogging solutions may exhibit various flaws, including poor performance and inability to interact and deploy easily. A need arises for a contemporary blog solution that is powerful and convenient to deploy. As such, this project intends to develop a functioning blog platform with likes, commenting, and uploading capabilities among other crucial features.

2. **Learning About Full-Stack Web Development**
   Full-stack web development requires comprehensive knowledge of multiple technologies, such as frontend development using Next.js and TypeScript and backend using FastAPI. Such skills are vital to gain experience on building complete web solutions.

3. **Handling Real-World Performance Problems**
   Common issues, like N+1 queries, may arise during the creation of database-driven applications and require immediate attention. This particular project deals with addressing this specific real-world problem.

4. **Docker Containerization**
   Application deployment in multiple environments usually results in inconsistency problems. However, through the process of application containerization with Docker and Docker Compose, it becomes possible to consistently deploy any application irrespective of the underlying machine, which is an important aspect in today's DevOps environment.

5. **Enhancing the Interactivity of the Site**
   Modern internet users are accustomed to interactive elements such as live notifications about likes, nesting comments, and instant uploading of images and videos. The current motivation for this project lies in developing a site that will allow users to be more than mere readers; they should be able to contribute to the conversation using threaded commenting and replying capabilities.

CHAPTER 3
OBJECTIVE

1. **Develop a Full-Fledged Blog Platform**

   To implement a fully-fledged blog application where registered users can create posts, see them, edit, and delete. It is important to make sure that the platform provides a convenient experience for both writers and visitors.

2. **Implement Secure Registration and Login**

   To create the registration and login mechanisms that would ensure the safety of user data on the website. Only registered users would have access to the functionality involving posting of any kind while others could just enjoy reading.

3. **Make Sure Likes Work Interactively**

   To make the like/unlike mechanism work and show real numbers of likes next to each post. Additionally, it is required to ensure that a user cannot like a post multiple times.

4. **Develop Nested Comments**

   To allow visitors of the website to not only leave new comments under posts but also respond to existing ones to start discussions.

5. **Provide Images Upload Capability**

   To ensure that users can upload images to the platform when creating their posts, commenting, and creating their personal profiles.

6. **Solve the N+1 Query Issue**

   To detect N+1 query problem and solve it to optimize database performance in terms of getting posts, comments, and likes.

7. **Dynamic routing should be implemented**

It will provide dynamic routes to be used for user profiles and each blog post. This will allow the users to get access to certain information using clean URLs.

8. **Containerize the application using Docker**

   This will help to containerize the frontend and backend parts of the application using Docker and Docker compose.

9. **Develop a responsive UI**

   A modern UI should be created using Next.js and Tailwind CSS.

10. **Design a RESTful API backend**

    A RESTful API backend should be designed using the FastAPI framework.

11. **Implement proper authorization and access control**

    Authorization should be added so that users can edit/delete their own posts, but no one else could do it.

12. **Document the development process**

    Proper documentation of the whole development process should be provided for the future use and training purposes.

CHAPTER 4

SCOPE

The Narrative Blog Platform Scope establishes the limits of the project, its capabilities, and deliverables. The scope indicates what is included in the current version and what may be considered in the future improvements.

Scope Includes:

User Management

- User registration with secure password hashing
- User login and logout functions
- User profile management
- JWT token-based sessions

Blog Post Management

- Ability to create blog posts with rich text
- Reading and viewing all blog posts on the home page
- Viewing individual blog posts with full information
- Ability to edit existing blog posts (author only)
- Ability to delete blog posts (author only)
- Supporting image uploading for blog posts

Likes System

- Ability to like a blog post (users who have signed up)
- Unliking the blog post
- Showing the number of likes in real time
- Restricting duplicate likes from one user

Commenting System

- Commenting on blog posts

- Nested replies capability
- Deleting own comments
- Displaying user information with the comment
- Thread comments for readability purposes

Image Upload

- Uploading images for blog posts
- Uploading images for comments
- Uploading profile images for users
- Storing and serving images through backend

Technical Implementation

- Using Next.js and TypeScript in frontend
- FastAPI in backend (Python-based)
- Using SQLite (development) / PostgreSQL (production) database
- Dockerized frontend and backend applications
- Docker Compose

Performance Optimization

- Handling the N+1 query problem
- Dynamic routing applied to profiles

Deployment

- Complete Docker Compose setup
- Frontend runs on port 3000
- Backend runs on port 8000
- Easy deployment on any machine with Docker installed

CHAPTER 5
INTENDED AUDIENCE

1. **Content Creators & Bloggers**: People interested in publishing their views, insights, and expertise on the internet in form of blog posts containing texts and images.

2. **General Audience**: Any reader who loves reading blogs and participating in discussions via commenting and replying.

3. **Students**: Computer Science and Engineering students studying full-stack web development, since the project provides a reference for implementing current technologies such as Next.js, FastAPI, and Docker.

4. **Software Engineers:** Developers seeking a reference for creating an entire blog platform that includes user authentication, CRUD functionality, liking mechanism, and nested comments.

5. **Enterprises**: Small firms or organizations that want to create an intranet-based blog portal for exchanging information within teams.

6. **Faculty Members:** Educators who aim to illustrate full-stack application development using real-world projects.

CHAPTER 6
FUNCTIONAL REQUIREMENTS

1. **User Authentication**

   Users have the ability to create an account and login. Only registered users can make changes such as updating and deleting information.

2. **Blogging**

   Registered users can create a new blog entry with images, read all blogs, get specific blog entry information, update their blog entries, and delete their blog entries**.**

3. **Like Functionality**

   Only registered users can like or dislike the blog entries. Real-time liking is supported, and users cannot like the same blog entry twice.

4. **Comments**

   Only registered users can make comments to the blog entries and reply to comments (nested replies). Registered users can delete their comments.

5. **Image Uploading**

   Users can upload images for the blog entry, comments, and profile picture. Images are uploaded and served from the backend.

6. **Authorization**

   Users can only edit and delete their entries. Visitors can only view the blogs and comments but not like them and comment on them**.**

7. Docker Containerized Application

The application is deployed using Docker Compose with the front-end on port 3000 and back-end on port 8000.

CHAPTER 7
NON-FUNCTIONAL REQUIREMENTS

7. **Performance**

   The application should handle multiple concurrent users efficiently. Database queries must be optimized to solve the N+1 problem, ensuring fast loading of posts and comments.

8. **Security**

   User passwords must be hashed before storing. Authentication tokens (JWT) should be secure and expire after a set time. Users can only access and modify their own content.

9. **Usability**

   The interface should be intuitive and easy to navigate. Users should be able to perform actions like creating posts, adding comments, and liking content without confusion.

10. **Reliability**

    The system must maintain data integrity. Duplicate likes from the same user on a single post should be prevented. Comments and posts should not be lost during concurrent operations**.**

11. **Scalability**

    The Dockerized architecture allows the application to scale easily. Additional containers can be added as user load increases.

12. **Portability**

    Since the application is containerized with Docker, it can run consistently on any machine Windows, macOS, or Linux without environment issues.

13. **Availability**

    The platform should be accessible whenever users need it. Backend APIs should respond to requests reliably**.**

14. **Maintainability**

    The code should be modular and well-documented, making it easy to add new features or fix bugs in the future.

15. **Compatibility**

    The frontend should work on modern web browsers including Chrome, Firefox, and Edge. The responsive design should work on desktops, tablets, and mobile devices.

CHAPTER 8

OPERATING ENVIRONMENT

1. HARDWARE REQUIREMENTS

Processor: Intel Core i3 or AMD equivalent (Recommended: Intel Core i5 or higher)
RAM: Minimum 4 GB (Recommended: 8 GB or higher)
Storage: 10 GB free disk space (Recommended: 20 GB free disk space)
Network: Internet connection for deployment

2. SOFTWARE REQUIREMENTS

Operating System: Windows 10/11, macOS 11+, or Ubuntu 20.04+
Docker: Version 20.10 or higher
Docker Compose: Version 2.0 or higher
Web Browser: Chrome, Firefox, or Edge (latest versions)

3. DEVELOPMENT ENVIRONMENT

Node.js: Version 18.x or higher
Python: Version 3.9 or higher
Code Editor: VS Code or PyCharm

4. DATABASE

Development: SQLite
Production: PostgreSQL

5. PORTS REQUIRED

Frontend (Next.js): Port 3000
Backend (FastAPI): Port 8000

CHAPTER 9

ARCHITECTURE DIAGRAM AND FLOWCHART

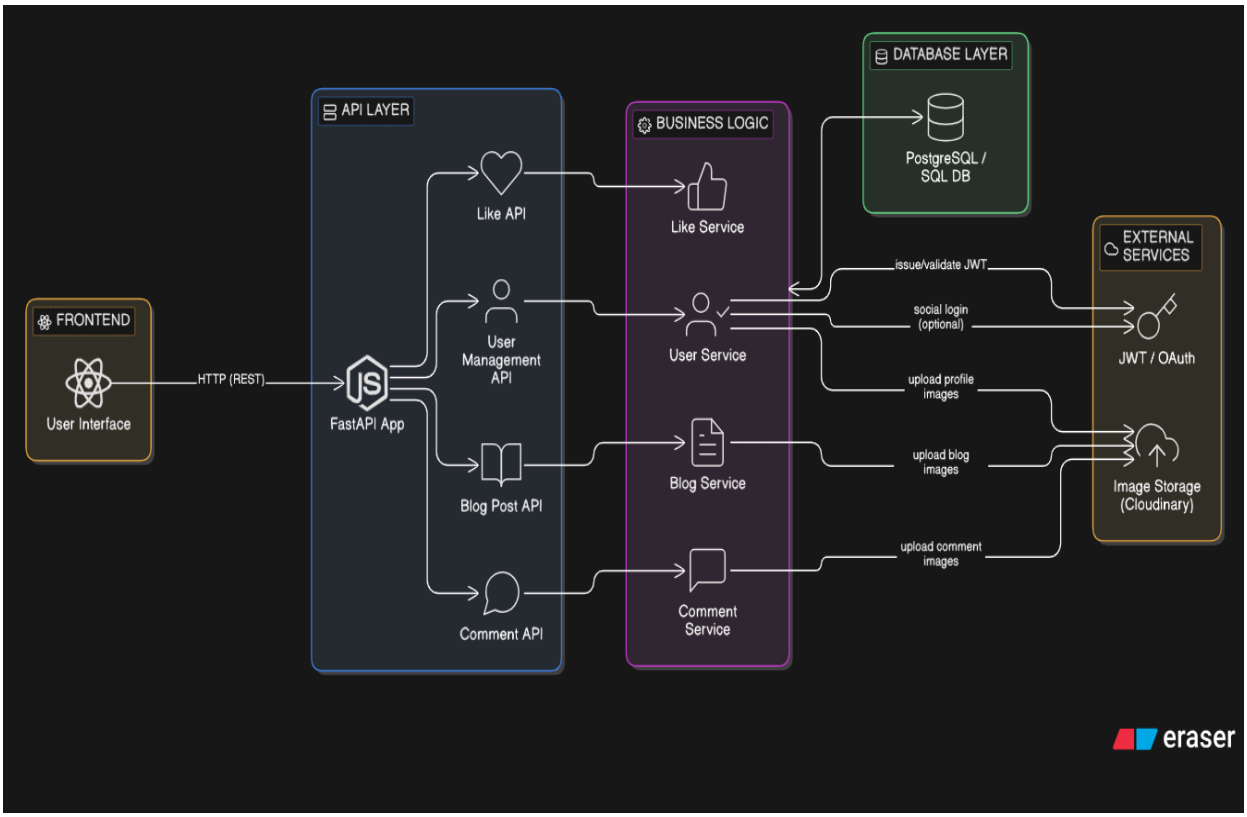

Fig. 1 High level Architecture Diagram

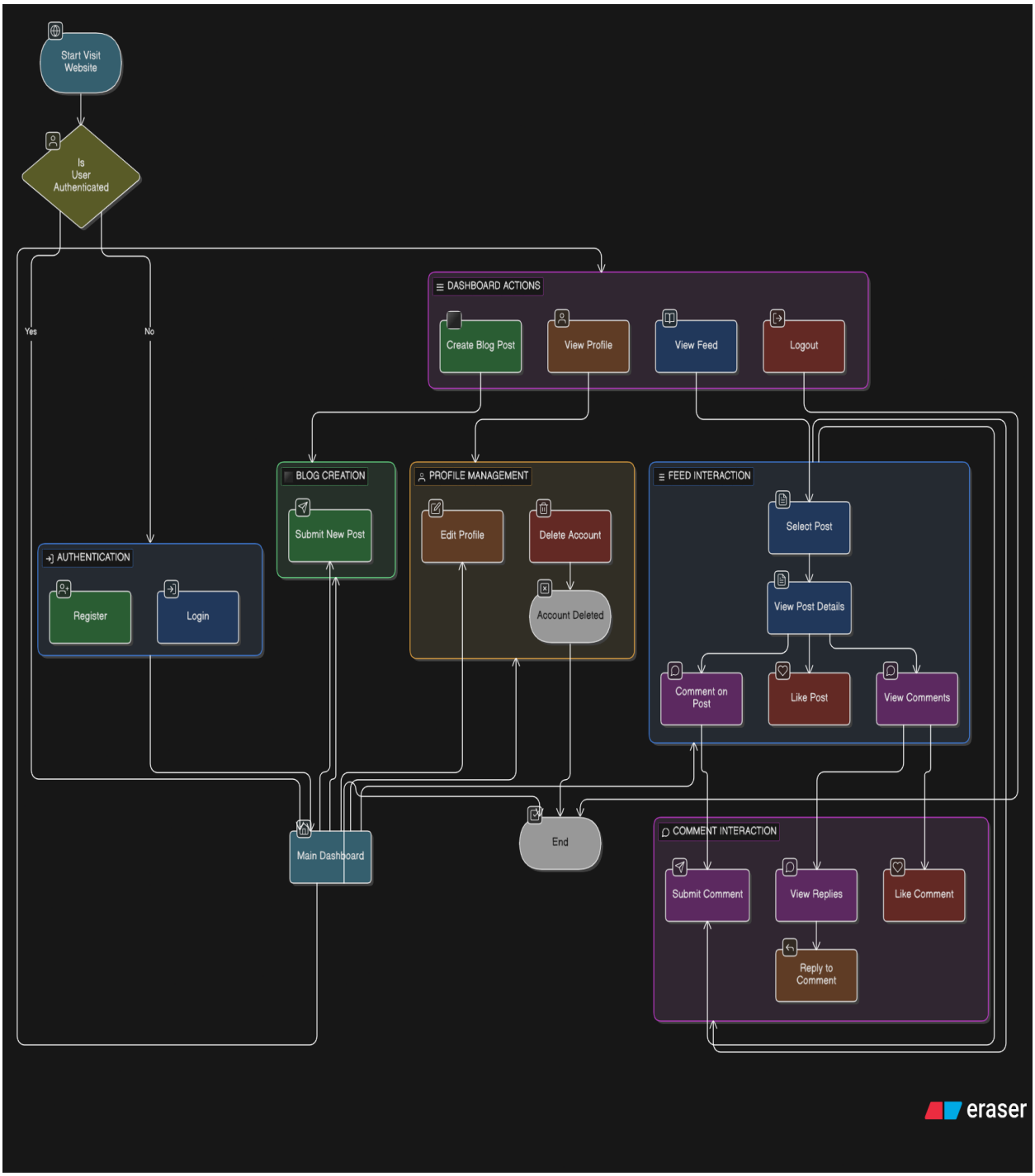

Fig. 2 Flowchart of Blog Platform

CHAPTER 10

IMPLEMENTATION DETAILS ALONG WITH SCREENSHOTS

10.1 Home Page

The Home Page serves as the entry point for all visitors. It features a gradient hero section with the platform branding, a value proposition, and prominent call-to-action buttons (Start Reading and Learn More). Below the hero, a real-time Analytics Dashboard is embedded showing Total Users and Total Visits with trend bar charts and a date-range filter.

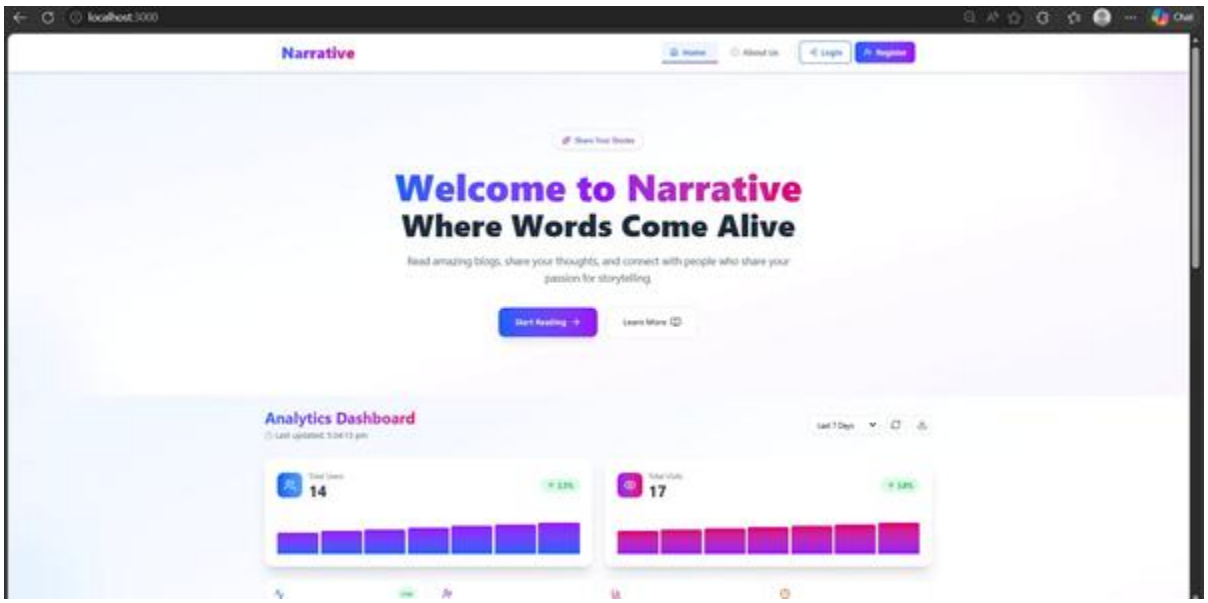

Fig. 1: Home Page - Hero section with 'Welcome to Narrative' and Analytics Dashboard

10.2 About Us Page

The About Us page introduces the development team behind the Narrative platform. It uses a card-based layout to present each team member with their role, specialization stack, and links to GitHub and LinkedIn profiles.

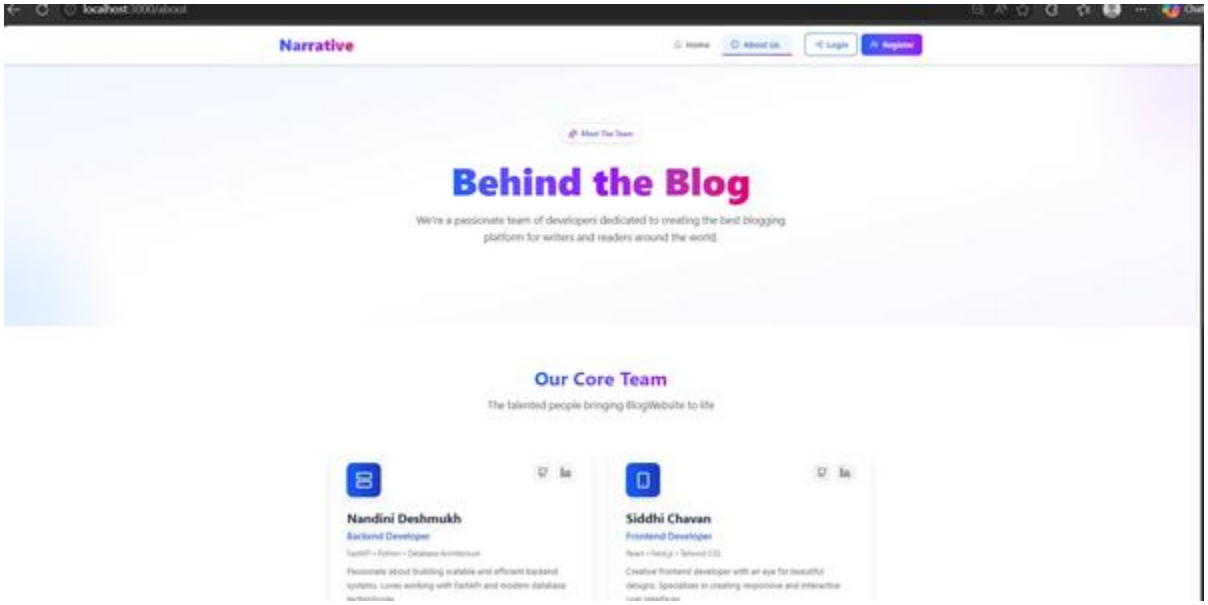

Fig. 2: About Us Page - Team profile cards for Nandini Deshmukh and Siddhi Chavan

10.3 Registration Page

The Registration page allows new users to sign up. It includes a circular profile photo upload preview, input fields for Username, Email, Password (with visibility toggle), and an optional Bio. A dropdown lets users indicate their intent (Read and Write Blogs). Users must agree to the Terms of Service and Privacy Policy before clicking Create Account.

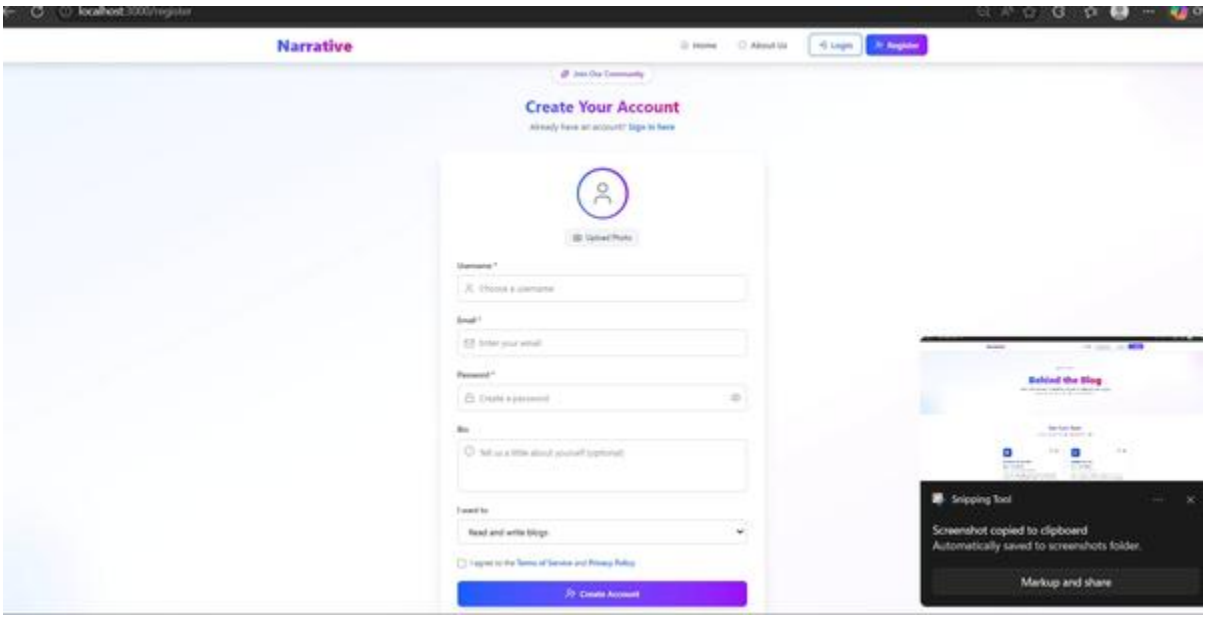

Fig. 3: Registration Page - Create Account form with photo upload and user details

10.4 Authenticated Feed

After logging in, users land on the personalized Feed. A left sidebar provides navigation links: Feed, Write Blog, My Blogs, Comments, Liked Posts, and Liked Comments. The user profile is shown at the bottom of the sidebar with a Logout button. The top bar includes a search input that can toggle between searching Users and Posts. Each post card displays the author avatar, username, timestamp, post title, excerpt, like count, and comment count.

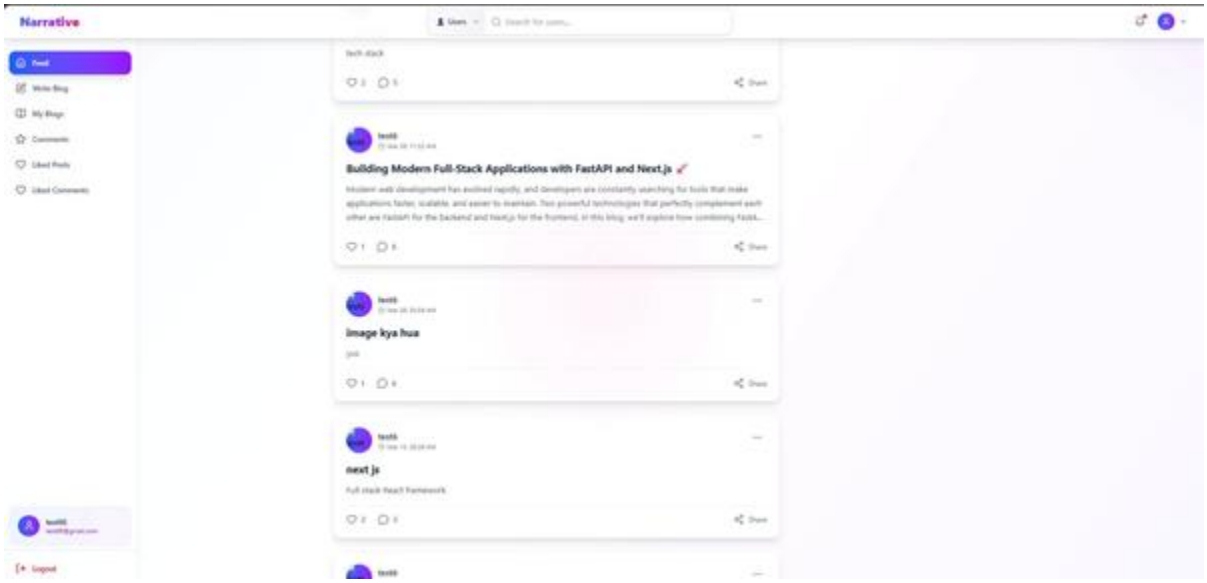

Fig. 4: Authenticated Feed - Blog post feed with sidebar navigation

10.5 Blog Post Detail Page

Clicking a post opens the full article in a dedicated page. A hero image area is shown at the top. Below it, the author avatar, name, publish date, and estimated read time are displayed alongside a bookmark icon. The article body renders with headings, paragraphs, and emoji formatting. A like count and comment count appear below the article body, followed by a Share button.

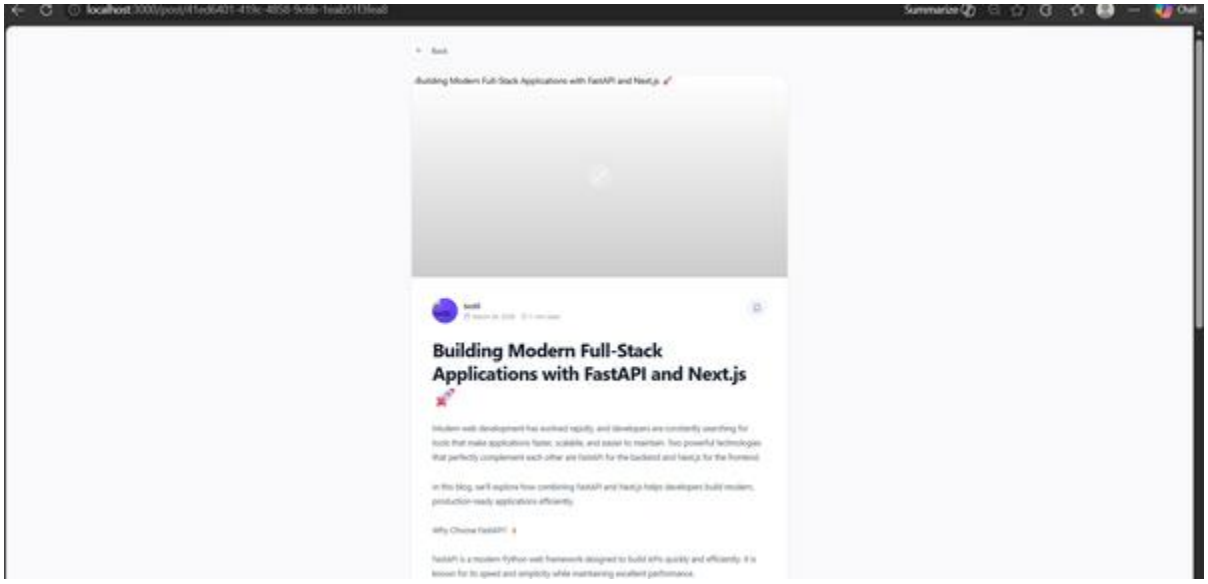

Fig. 5: Blog Post Detail - Full article view with title, content, and author info

10.6 Comments Section

The Comments Section is displayed at the bottom of each blog post page. Users can write a comment in a text input area and optionally attach an image via Choose File. Existing comments show the commenter's avatar, username, timestamp, and comment text. Each comment supports a Like button, a Reply link for nested replies, and a Show 1 reply toggle to expand threaded discussions.

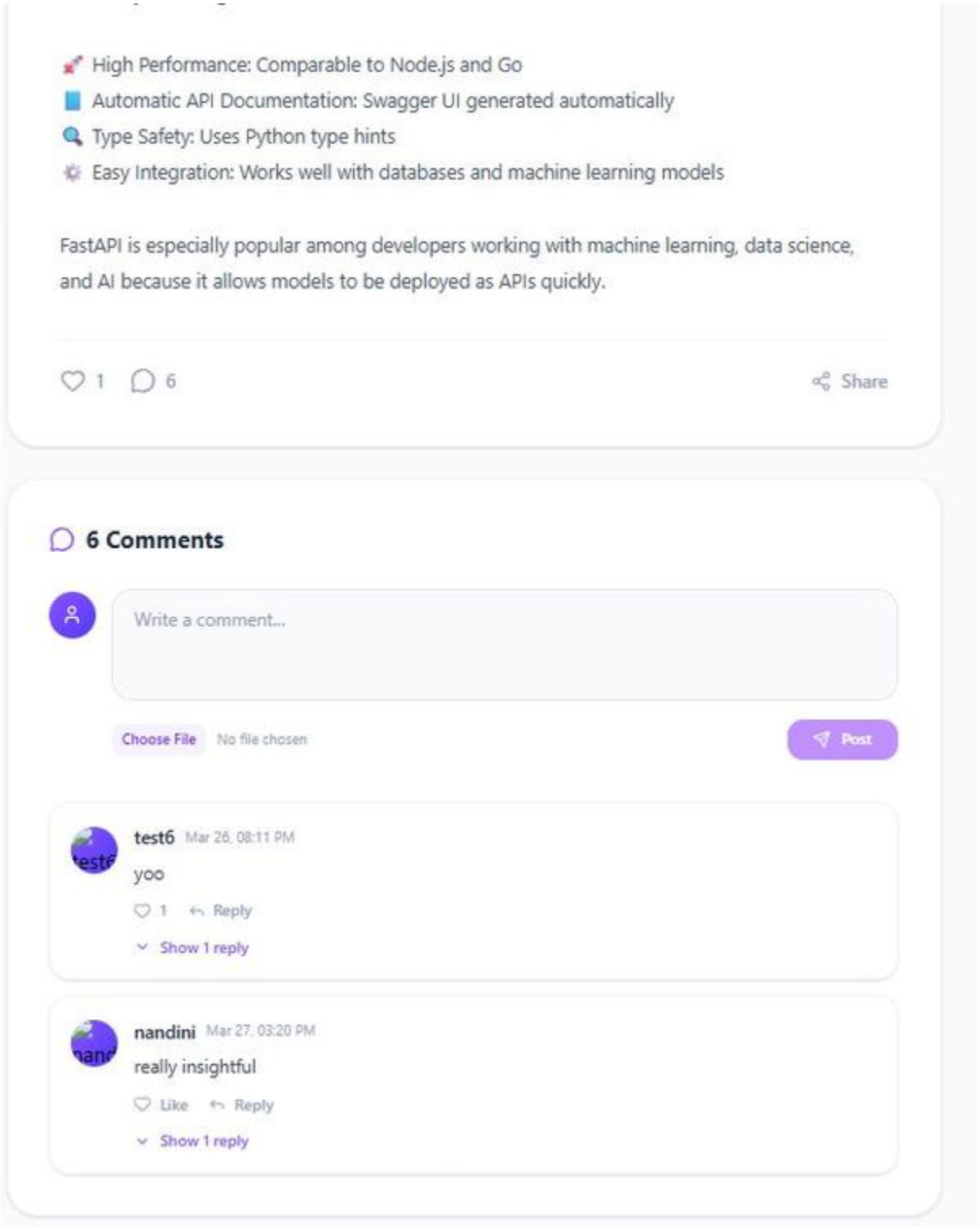

Fig. 6: Comments Section - Nested comments with like and reply functionality

CHAPTER 11
CONCLUSION

The Narrative Blog Platform mini-project has successfully achieved its primary objectives of developing a full-stack, feature-rich blog application using modern web technologies.

1. **Summary of Achievements:**

   The project successfully implemented all planned features including secure user authentication, complete CRUD operations for blog posts, an interactive likes system with duplicate prevention, a nested commenting system supporting threaded replies, and image upload functionality for posts, comments, and user profiles.

2. **Technical Accomplishments:**

   From a technical perspective, the project addressed critical performance challenges such as the N+1 query problem, ensuring efficient database operations. Dynamic routing was implemented for user profiles and individual blog posts, providing clean and meaningful URLs. The entire application was containerized using Docker and Docker Compose, enabling consistent deployment across any machine without environment-related issues.

3. **Technology Stack Validation:**

   The chosen technology stack proved to be effective. Next.js with TypeScript provided a robust and type-safe frontend development experience. FastAPI delivered high-performance REST API endpoints with automatic validation. Docker simplified the deployment process, allowing both frontend (port 3000) and backend (port 8000) services to run seamlessly together.

4. **Learning Outcomes:**

This project provided valuable hands-on experience in full-stack web development. Key learning areas include implementing JWT-based authentication, designing RESTful APIs, managing database relationships with SQLAlchemy, creating responsive user interfaces with Tailwind CSS, and containerizing applications for production deployment.

5. **Limitations:**

   The current version lacks certain features such as real-time notifications, search functionality, email verification, and post categories. These have been identified as future enhancements.

6. **Future Work:**

   Planned enhancements include integrating Kafka for real-time notifications, implementing Redis for caching frequently accessed data, adding advanced search capabilities, and developing an admin dashboard for user and content management.

7. **Final Verdict:**

   The Narrative Blog Platform serves as a complete, production-ready reference implementation of a modern blog system. It demonstrates best practices in full-stack development, performance optimization, and Docker-based deployment. The project successfully meets all requirements outlined at the outset and provides a solid foundation for future extensions.

CHPATER 12
REFERENCES

1. Documentation of **Next.js.** "Next.js Framework Documentation." Available at: https://nextjs.org/docs

2. Documentation of **FastAPI**. "FastAPI Framework Documentation." Available at: https://fastapi.tiangolo.com/

3. Documentation of **Docker**. "Docker Container Platform Documentation." Available at: https://docs.docker.com/

4. Documentation of **Docker Compose.** "Docker Compose Orchestration Documentation." Available at: https://docs.docker.com/compose/

5. Documentation of TypeScript. "TypeScript Programming Language Documentation." Available at: https://www.typescriptlang.org/docs/

6. Documentation of Tailwind CSS. "Tailwind CSS Framework Documentation." Available at: https://tailwindcss.com/docs

7. Documentation of SQLAlchemy. "SQLAlchemy ORM Documentation." Available at: https://www.sqlalchemy.org/

8. Documentation of React. "React JavaScript Library Documentation." Available at: https://react.dev/

9. JWT.io. "JSO
N Web Tokens Introduction and Documentation." Available at: https://jwt.io/

10. Cloudinary Documentation. "Cloudinary Image and Video Management Platform." Available at: https://cloudinary.com/documentation

11. Documentation of Python. "Python Programming Language Documentation." Available at: https://docs.python.org/3/

12. Documentation of Node.js. "Node.js JavaScript Runtime Documentation." Available at: https://nodejs.org/en/docs/

13. Documentation of SQLite. "SQLite Database Documentation." Available at: https://www.sqlite.org/docs.html

14. Codevolution. "Next.js 15 Full course One shot - Complete Playlist (App Router)." Available at: https://www.youtube.com/watch?v=ZjAqacIC_3c&list=PLC3y8-rFHvwjOKd6gdf4QtV1uYNiQnruI